

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

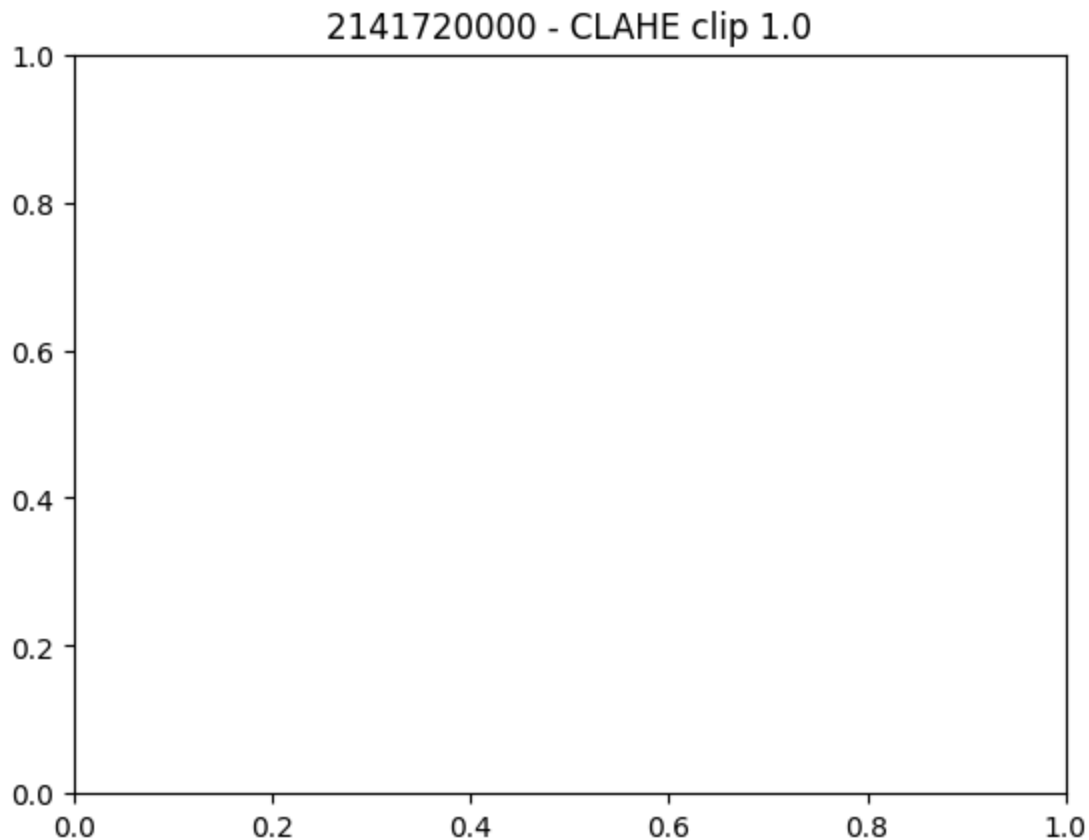
```
import cv2 as cv
from google.colab.patches import cv2_imshow
from skimage import io
import matplotlib.pyplot as plt
import numpy as np
import math
import os
import glob
```

```
# ===== SEL IDENTITAS - JANGAN DIHAPUS, HARUS SEL PERTAMA =====
import sys, platform, hashlib, datetime, zoneinfo
NAMA = 'Tulis nama lengkap Anda'
NIM  = '2141720000'          # ganti dengan NIM Anda
N    = int(NIM[-3:])
PARAM = {
    'bins' : 2 ** ((N % 4) + 3),
    'clip' : round(1.0 + (N % 5) * 0.5, 1),
    'tile' : (N % 4) + 2,
    'L'    : 2 ** ((N % 3) + 1),
}
waktu = datetime.datetime.now(zoneinfo.ZoneInfo('Asia/Jakarta'))
print('NAMA   :', NAMA)
print('NIM    :', NIM, '| N =', N)
for k, v in PARAM.items():
    print('%-6s : ' % k, v)
print('WAKTU   :', waktu.strftime('%Y-%m-%d %H:%M:%S %Z'))
print('SISTEM  :', sys.version.split()[0], platform.platform())
kunci = NIM + '|' + waktu.strftime('%Y-%m-%d')
print('TOKEN   :', hashlib.sha256(kunci.encode()).hexdigest()[:16])
plt.title(NIM + ' - CLAHE clip ' + str(PARAM['clip']))
```

```

NAMA   : Tulis nama lengkap Anda
NIM    : 2141720000 | N = 0
bins   : 8
clip   : 1.0
tile   : 2
L      : 2
WAKTU  : 2026-09-22 14:09:37 WIB
SISTEM : 3.13.15 Linux-6.6.122+-x86_64-with-glibc2.39
TOKEN  : 4d1a30820e109a1c
Text(0.5, 1.0, '2141720000 - CLAHE clip 1.0')

```



```

from re import S
#membuat histogram image (manual)
img = cv.imread('/content/drive/MyDrive/PCVK/Images/lena.jpg')
img = cv.cvtColor(img, cv.COLOR_BGR2RGB)

height, width, depth = np.shape(img)
names = np.arange(256)

red = [0]*256
green = [0]*256
blue = [0]*256

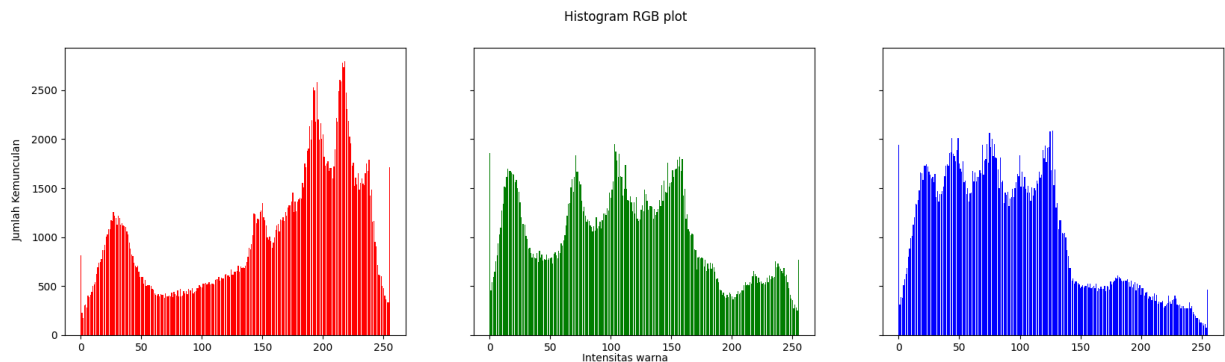
for y in range(height):
    for x in range(width):
        red[img[y][x][0]] += 1
        green[img[y][x][1]] += 1
        blue[img[y][x][2]] += 1

```

```

names = np.arange(256)
fig, axs = plt.subplots(1, 3, figsize=[20,5], sharex =True, sharey=True)
fig.suptitle('Histogram RGB plot')
fig.text(0.09, 0.5, 'Jumlah Kemunculan', va='center',rotation='vertical')
fig.text(0.5,0.04,'Intensitas warna', ha='center')
axs[0].bar(names, red,color='red')
axs[1].bar(names, green,color='green')
axs[2].bar(names, blue,color='blue')
plt.show()

```



Pertanyaan Praktikum E1

1. Buatlah histogram citra yang sama menggunakan library NumPy, yaitu `numpy.histogram`, dan juga `cv.calcHist`. Bandingkan ketiganya dengan menghitung selisih maksimum antar hasil. Selisihnya harus nol; tampilkan pembuktiannya pada `lena.jpg` lebih dulu, kemudian ulangi pada `KTM1b.jpg`.
2. Gambarkan histogram ternormalisasi $p(j)$ dan kurva CDF untuk `KTM1a.jpg` sampai `KTM1d.jpg` dalam satu figure 2×2 . Jelaskan perbedaan bentuk kurva CDF antara citra paling gelap dan citra paling terang milik Anda, lalu kaitkan dengan kondisi pencahayaan saat akuisisi pada Pertemuan 2.
3. Ulangi histogram `KTM1b.jpg` dengan jumlah bin sebesar `PARAM['bins']` dan dengan 256 bin. Tampilkan keduanya berdampingan dan jelaskan informasi apa yang hilang ketika jumlah bin dikurangi.

Jawaban !

✓ Jawaban E1 - Soal 1 Membuat Histogram Citra

Double-click (or enter) to edit

```

import matplotlib.pyplot as plt
import numpy as np
import cv2 as cv

# Assuming lena_rgb and ktm1b_rgb are already loaded and processed in previous
# And hist_np_x_lena/ktm and hist_cv_x_lena/ktm are available from previous cal

# --- Plotting for Lena.jpg ---
print("\n--- Visualizing Histograms for lena.jpg ---")
fig, axs = plt.subplots(2, 3, figsize=(18, 10))
fig.suptitle('Histograms for Lena.jpg (NumPy vs OpenCV)', fontsize=16)

# Red Channel
axs[0, 0].bar(np.arange(256), hist_np_r_lena, color='red', width=1, label='NumPy')
axs[0, 0].set_title('Lena.jpg - Red Channel (NumPy)')
axs[0, 0].legend()
axs[1, 0].bar(np.arange(256), hist_cv_r_lena, color='red', width=1, label='OpenCV')
axs[1, 0].set_title('Lena.jpg - Red Channel (OpenCV)')
axs[1, 0].legend()

# Green Channel
axs[0, 1].bar(np.arange(256), hist_np_g_lena, color='green', width=1, label='NumPy')
axs[0, 1].set_title('Lena.jpg - Green Channel (NumPy)')
axs[0, 1].legend()
axs[1, 1].bar(np.arange(256), hist_cv_g_lena, color='green', width=1, label='OpenCV')
axs[1, 1].set_title('Lena.jpg - Green Channel (OpenCV)')
axs[1, 1].legend()

# Blue Channel
axs[0, 2].bar(np.arange(256), hist_np_b_lena, color='blue', width=1, label='NumPy')
axs[0, 2].set_title('Lena.jpg - Blue Channel (NumPy)')
axs[0, 2].legend()
axs[1, 2].bar(np.arange(256), hist_cv_b_lena, color='blue', width=1, label='OpenCV')
axs[1, 2].set_title('Lena.jpg - Blue Channel (OpenCV)')
axs[1, 2].legend()

plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()

# --- Plotting for KTM1b.jpg ---
print("\n--- Visualizing Histograms for KTM1b.jpg ---")
fig, axs = plt.subplots(2, 3, figsize=(18, 10))
fig.suptitle('Histograms for KTM1b.jpg (NumPy vs OpenCV)', fontsize=16)

# Red Channel
axs[0, 0].bar(np.arange(256), hist_np_r_ktm, color='red', width=1, label='NumPy')
axs[0, 0].set_title('KTM1b.jpg - Red Channel (NumPy)')
axs[0, 0].legend()
axs[1, 0].bar(np.arange(256), hist_cv_r_ktm, color='red', width=1, label='OpenCV')
axs[1, 0].set_title('KTM1b.jpg - Red Channel (OpenCV)')
axs[1, 0].legend()

```

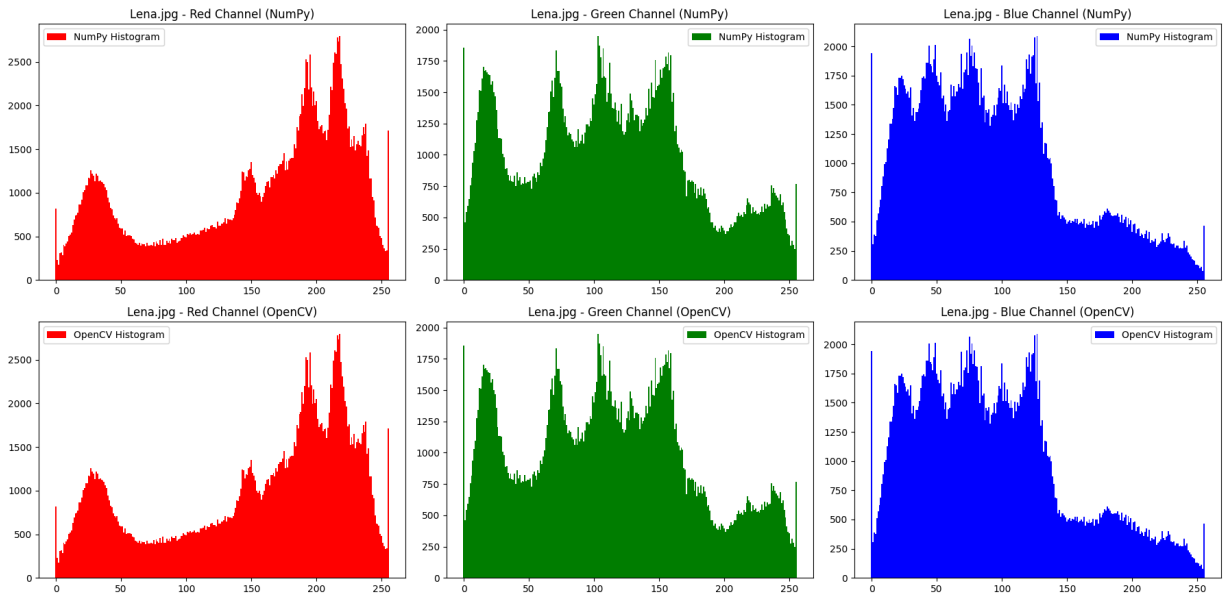
```
# Green Channel
axs[0, 1].bar(np.arange(256), hist_np_g_ktm, color='green', width=1, label='NumPy')
axs[0, 1].set_title('KTM1b.jpg - Green Channel (NumPy)')
axs[0, 1].legend()
axs[1, 1].bar(np.arange(256), hist_cv_g_ktm, color='green', width=1, label='OpenCV')
axs[1, 1].set_title('KTM1b.jpg - Green Channel (OpenCV)')
axs[1, 1].legend()

# Blue Channel
axs[0, 2].bar(np.arange(256), hist_np_b_ktm, color='blue', width=1, label='NumPy')
axs[0, 2].set_title('KTM1b.jpg - Blue Channel (NumPy)')
axs[0, 2].legend()
axs[1, 2].bar(np.arange(256), hist_cv_b_ktm, color='blue', width=1, label='OpenCV')
axs[1, 2].set_title('KTM1b.jpg - Blue Channel (OpenCV)')
axs[1, 2].legend()

plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()
```

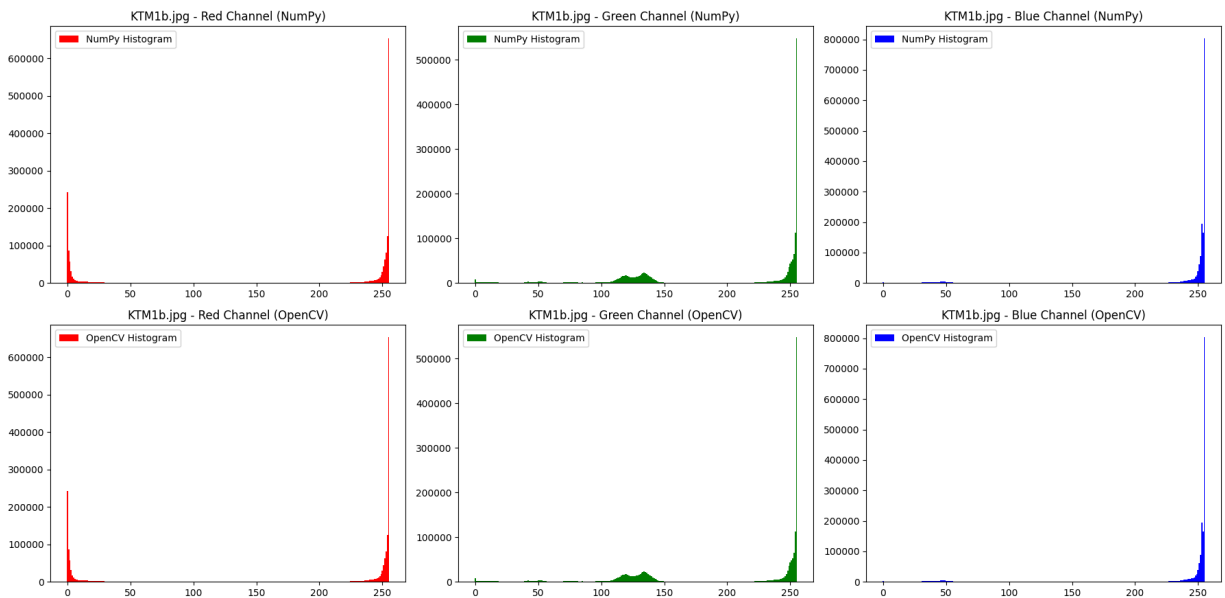
--- Visualizing Histograms for lena.jpg ---

Histograms for Lena.jpg (NumPy vs OpenCV)



--- Visualizing Histograms for KTM1b.jpg ---

Histograms for KTM1b.jpg (NumPy vs OpenCV)



```
import cv2 as cv
import numpy as np
```

```
# Define a helper function to calculate manual histogram for RGB image
def calculate_manual_histogram(image_rgb):
    height, width, _ = image_rgb.shape
    hist_r = np.zeros(256, dtype=int)
    hist_g = np.zeros(256, dtype=int)
    hist_b = np.zeros(256, dtype=int)
```

```

    for y in range(height):
        for x in range(width):
            hist_r[image_rgb[y, x, 0]] += 1
            hist_g[image_rgb[y, x, 1]] += 1
            hist_b[image_rgb[y, x, 2]] += 1
    return hist_r, hist_g, hist_b

# --- Processing for lena.jpg ---
print("--- Comparing Histograms for lena.jpg ---")

# Load lena.jpg and convert to RGB for consistent histogram calculation
lena_bgr = cv.imread('/content/drive/MyDrive/PCVK/Images/lena.jpg')
lena_rgb = cv.cvtColor(lena_bgr, cv.COLOR_BGR2RGB)

# 1. Manual Histogram for lena.jpg
manual_hist_r_lena, manual_hist_g_lena, manual_hist_b_lena = calculate_manual_hist

# 2. NumPy Histogram for lena.jpg
hist_np_r_lena = np.histogram(lena_rgb[:, :, 0], bins=256, range=[0, 256])[0]
hist_np_g_lena = np.histogram(lena_rgb[:, :, 1], bins=256, range=[0, 256])[0]
hist_np_b_lena = np.histogram(lena_rgb[:, :, 2], bins=256, range=[0, 256])[0]

# 3. OpenCV Histogram for lena.jpg
hist_cv_r_lena = cv.calcHist([lena_rgb], [0], None, [256], [0, 256]).flatten()
hist_cv_g_lena = cv.calcHist([lena_rgb], [1], None, [256], [0, 256]).flatten()
hist_cv_b_lena = cv.calcHist([lena_rgb], [2], None, [256], [0, 256]).flatten()

# Compare and print differences for Lena.jpg
print("\nLena.jpg Differences:")
print("Red Channel:")
print(f"  Max difference (Manual vs NumPy): {np.max(np.abs(manual_hist_r_lena - hist_np_r_lena))}")
print(f"  Max difference (Manual vs OpenCV): {np.max(np.abs(manual_hist_r_lena - hist_cv_r_lena))}")
print(f"  Max difference (NumPy vs OpenCV): {np.max(np.abs(hist_np_r_lena - hist_cv_r_lena))}")

print("Green Channel:")
print(f"  Max difference (Manual vs NumPy): {np.max(np.abs(manual_hist_g_lena - hist_np_g_lena))}")
print(f"  Max difference (Manual vs OpenCV): {np.max(np.abs(manual_hist_g_lena - hist_cv_g_lena))}")
print(f"  Max difference (NumPy vs OpenCV): {np.max(np.abs(hist_np_g_lena - hist_cv_g_lena))}")

print("Blue Channel:")
print(f"  Max difference (Manual vs NumPy): {np.max(np.abs(manual_hist_b_lena - hist_np_b_lena))}")
print(f"  Max difference (Manual vs OpenCV): {np.max(np.abs(manual_hist_b_lena - hist_cv_b_lena))}")
print(f"  Max difference (NumPy vs OpenCV): {np.max(np.abs(hist_np_b_lena - hist_cv_b_lena))}")

# --- Processing for KTM1b.jpg ---
print("\n--- Comparing Histograms for KTM1b.jpg ---")

# Load KTM1b.jpg and convert to RGB for consistent histogram calculation
ktm1b_bgr = cv.imread('/content/drive/MyDrive/PCVK/Images/244107020064/KTM1b.jpg')
ktm1b_rgb = cv.cvtColor(ktm1b_bgr, cv.COLOR_BGR2RGB)

```

```
# 1. Manual Histogram for KTM1b.jpg
manual_hist_r_ktm, manual_hist_g_ktm, manual_hist_b_ktm = calculate_manual_hist

# 2. NumPy Histogram for KTM1b.jpg
hist_np_r_ktm = np.histogram(ktm1b_rgb[:, :, 0], bins=256, range=[0, 256])[0]
hist_np_g_ktm = np.histogram(ktm1b_rgb[:, :, 1], bins=256, range=[0, 256])[0]
hist_np_b_ktm = np.histogram(ktm1b_rgb[:, :, 2], bins=256, range=[0, 256])[0]

# 3. OpenCV Histogram for KTM1b.jpg
hist_cv_r_ktm = cv.calcHist([ktm1b_rgb], [0], None, [256], [0, 256]).flatten()
hist_cv_g_ktm = cv.calcHist([ktm1b_rgb], [1], None, [256], [0, 256]).flatten()
hist_cv_b_ktm = cv.calcHist([ktm1b_rgb], [2], None, [256], [0, 256]).flatten()

# Compare and print differences for KTM1b.jpg
print("\nKTM1b.jpg Differences:")
print("Red Channel:")
print(f"  Max difference (Manual vs NumPy): {np.max(np.abs(manual_hist_r_ktm - hist_np_r_ktm))}")
print(f"  Max difference (Manual vs OpenCV): {np.max(np.abs(manual_hist_r_ktm - hist_cv_r_ktm))}")
print(f"  Max difference (NumPy vs OpenCV): {np.max(np.abs(hist_np_r_ktm - hist_cv_r_ktm))}")

print("Green Channel:")
print(f"  Max difference (Manual vs NumPy): {np.max(np.abs(manual_hist_g_ktm - hist_np_g_ktm))}")
print(f"  Max difference (Manual vs OpenCV): {np.max(np.abs(manual_hist_g_ktm - hist_cv_g_ktm))}")
print(f"  Max difference (NumPy vs OpenCV): {np.max(np.abs(hist_np_g_ktm - hist_cv_g_ktm))}")

print("Blue Channel:")
print(f"  Max difference (Manual vs NumPy): {np.max(np.abs(manual_hist_b_ktm - hist_np_b_ktm))}")
print(f"  Max difference (Manual vs OpenCV): {np.max(np.abs(manual_hist_b_ktm - hist_cv_b_ktm))}")
print(f"  Max difference (NumPy vs OpenCV): {np.max(np.abs(hist_np_b_ktm - hist_cv_b_ktm))}")
```

--- Comparing Histograms for lena.jpg ---

Lena.jpg Differences:

Red Channel:

```
  Max difference (Manual vs NumPy): 0
  Max difference (Manual vs OpenCV): 0.0
  Max difference (NumPy vs OpenCV): 0.0
```

Green Channel:

```
  Max difference (Manual vs NumPy): 0
  Max difference (Manual vs OpenCV): 0.0
  Max difference (NumPy vs OpenCV): 0.0
```

Blue Channel:

```
  Max difference (Manual vs NumPy): 0
  Max difference (Manual vs OpenCV): 0.0
  Max difference (NumPy vs OpenCV): 0.0
```

--- Comparing Histograms for KTM1b.jpg ---

KTM1b.jpg Differences:

Red Channel:

```
  Max difference (Manual vs NumPy): 0
  Max difference (Manual vs OpenCV): 0.0
```

```

Max difference (NumPy vs OpenCV): 0.0
Green Channel:
Max difference (Manual vs NumPy): 0
Max difference (Manual vs OpenCV): 0.0
Max difference (NumPy vs OpenCV): 0.0
Blue Channel:
Max difference (Manual vs NumPy): 0
Max difference (Manual vs OpenCV): 0.0
Max difference (NumPy vs OpenCV): 0.0

```

Jawaban Praktikum E1 - Soal 2: Histogram Ternormalisasi dan Kurva CDF

```

import matplotlib.pyplot as plt
import numpy as np
import cv2 as cv
import os

# Prepare to store processed data
images_gray = {}
normalized_hists = {}
cdfs = {}

# Load KTM images
base_path = '/content/drive/MyDrive/PCVK/Images/244107020064/'
ktm_image_files = ['KTM1a.jpg', 'KTM1b.jpg', 'KTM1c.jpg', 'KTM1d.jpg']
ktm_images = {}
for f in ktm_image_files:
    path = os.path.join(base_path, f)
    img = cv.imread(path)
    if img is None:
        print(f"Warning: Could not load image {f} from {path}. Skipping.")
        continue
    ktm_images[f] = img

for name, img_data in ktm_images.items():
    if img_data is None:
        print(f"Skipping processing for {name} due to loading error.")
        continue

    # Check number of channels in the image
    if img_data.ndim == 3 and img_data.shape[2] == 3: # If it's a 3-channel BGR
        gray_img = cv.cvtColor(img_data, cv.COLOR_BGR2GRAY)
    elif img_data.ndim == 2: # If it's already a grayscale image (2 dimensions)
        gray_img = img_data
    else:
        print(f"Warning: Image {name} has an unexpected number of channels/dime
        continue

```

```

images_gray[name] = gray_img

# Calculate histogram
hist = cv.calcHist([gray_img], [0], None, [256], [0, 256])

# Normalize histogram
normalized_hist = hist / (gray_img.shape[0] * gray_img.shape[1])
normalized_hists[name] = normalized_hist

# Calculate CDF (Cumulative Distribution Function)
cdf = normalized_hist.cumsum()
cdfs[name] = cdf

print("Histograms and CDFs calculated for all KTM images.")

```

Histograms and CDFs calculated for all KTM images.

```

fig, axes = plt.subplots(2, 2, figsize=(15, 12))
fig.suptitle('Normalized Histograms and CDFs for KTM Images', fontsize=16)

# Flatten axes for easy iteration
axes = axes.flatten()

image_names_sorted = sorted(ktm_images.keys()) # Sort names for consistent order

for i, name in enumerate(image_names_sorted):
    ax = axes[i]

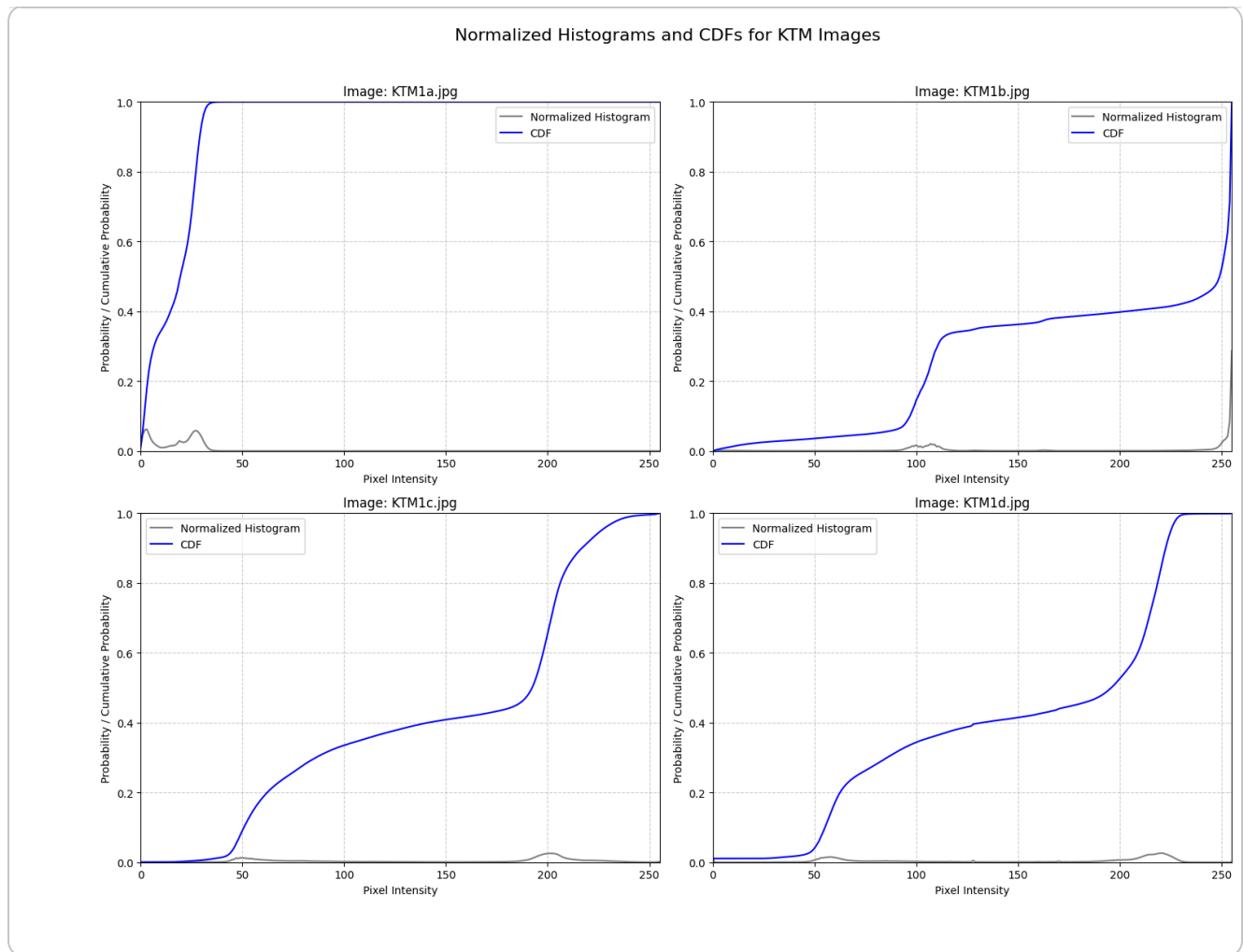
    # Plot Normalized Histogram
    ax.plot(normalized_hists[name], color='gray', label='Normalized Histogram')

    # Plot CDF
    ax.plot(cdfs[name], color='blue', label='CDF')

    ax.set_title(f'Image: {name}', fontsize=12)
    ax.set_xlabel('Pixel Intensity', fontsize=10)
    ax.set_ylabel('Probability / Cumulative Probability', fontsize=10)
    ax.legend()
    ax.grid(True, linestyle='--', alpha=0.6)
    ax.set_xlim([0, 255])
    ax.set_ylim([0, 1])

plt.tight_layout(rect=[0, 0.03, 1, 0.96]) # Adjust layout to prevent title overlap
plt.show()

```



Penjelasan Perbedaan Kurva CDF

Berdasarkan plot di atas dan nilai kecerahan rata-rata (`image_brightness`) yang telah dihitung sebelumnya:

- **Gambar paling gelap (KTM1a.jpg)**: Kurva CDF-nya cenderung naik dengan cepat di awal (intensitas pixel rendah) dan kemudian mendatar. Ini menunjukkan bahwa sebagian besar piksel dalam gambar memiliki nilai intensitas yang rendah (gelap). Kenaikan yang curam di bagian awal mengindikasikan bahwa banyak piksel terkonsentrasi pada rentang gelap, dan mencapai nilai kumulatif 1 (atau mendekati 1) jauh sebelum mencapai intensitas piksel maksimum (255).
- **Gambar paling terang (KTM1b.jpg)**: Sebaliknya, kurva CDF untuk gambar ini akan menunjukkan kenaikan yang lebih lambat di awal dan akan mulai menanjak tajam pada intensitas piksel yang lebih tinggi. Ini mengindikasikan bahwa distribusi piksel didominasi oleh nilai intensitas yang tinggi (cerah), dan kurva membutuhkan waktu lebih lama untuk mencapai nilai kumulatif 1 karena piksel-piksennya tersebar di rentang yang lebih cerah.

Kaitan dengan Kondisi Pencahayaan saat Akuisisi:

- **Citra paling gelap (KTM1a.jpg)** memiliki kurva CDF yang bergeser ke kiri, menandakan mayoritas pikselnya gelap. Ini kemungkinan besar disebabkan oleh kondisi pencahayaan yang **kurang memadai atau redup** saat gambar diambil. Objek dalam citra mungkin kurang terpapar cahaya, sehingga menghasilkan detail yang tersembunyi di area bayangan dan kontras yang rendah.
- **Citra paling terang (KTM1b.jpg)** memiliki kurva CDF yang bergeser ke kanan, menandakan mayoritas pikselnya cerah. Ini menunjukkan bahwa gambar diambil dalam kondisi pencahayaan yang **sangat terang atau berlimpah**. Hal ini bisa menyebabkan beberapa area terang menjadi 'overexposed' (terlalu cerah) sehingga kehilangan detail, atau secara umum, citra memiliki distribusi warna yang cenderung ke arah putih/terang.

Jawaban Praktikum E1 - Soal 3: Perbandingan Histogram dengan Jumlah Bin Berbeda

```
import matplotlib.pyplot as plt
import numpy as np
import cv2 as cv

# Ensure PARAM dictionary is defined. If not, define it for this example.
if 'PARAM' not in locals():
    PARAM = {'bins': 64} # Default value if not already set

# Assuming ktm_images and images_gray are available from previous cells
image_name = 'KTM1b.jpg'

# Get the grayscale image for KTM1b.jpg
if image_name in images_gray:
    ktm1b_gray = images_gray[image_name]
else:
    # Fallback if images_gray wasn't populated or KTM1b.jpg was skipped
    print(f"Reloading {image_name} for processing.")
    base_path = '/content/drive/MyDrive/PCVK/Images/244107020064/'
    ktm1b_bgr = cv.imread(os.path.join(base_path, image_name))
    if ktm1b_bgr is not None:
        ktm1b_gray = cv.cvtColor(ktm1b_bgr, cv.COLOR_BGR2GRAY)
    else:
        print(f"Error: Could not load {image_name}. Skipping histogram comparis
        ktm1b_gray = None

if ktm1b_gray is not None:
    # Histogram with 256 bins
```

```

hist_256_bins = cv.calcHist([ktm1b_gray], [0], None, [256], [0, 256]).flatten()

# Histogram with PARAM['bins'] bins
num_bins_param = PARAM['bins']
hist_param_bins = cv.calcHist([ktm1b_gray], [0], None, [num_bins_param], [0, 256])

# Create bin edges and centers for plotting
bin_edges_256 = np.arange(257)
bin_centers_256 = (bin_edges_256[:-1] + bin_edges_256[1:]) / 2

bin_edges_param = np.linspace(0, 256, num_bins_param + 1)
bin_centers_param = (bin_edges_param[:-1] + bin_edges_param[1:]) / 2

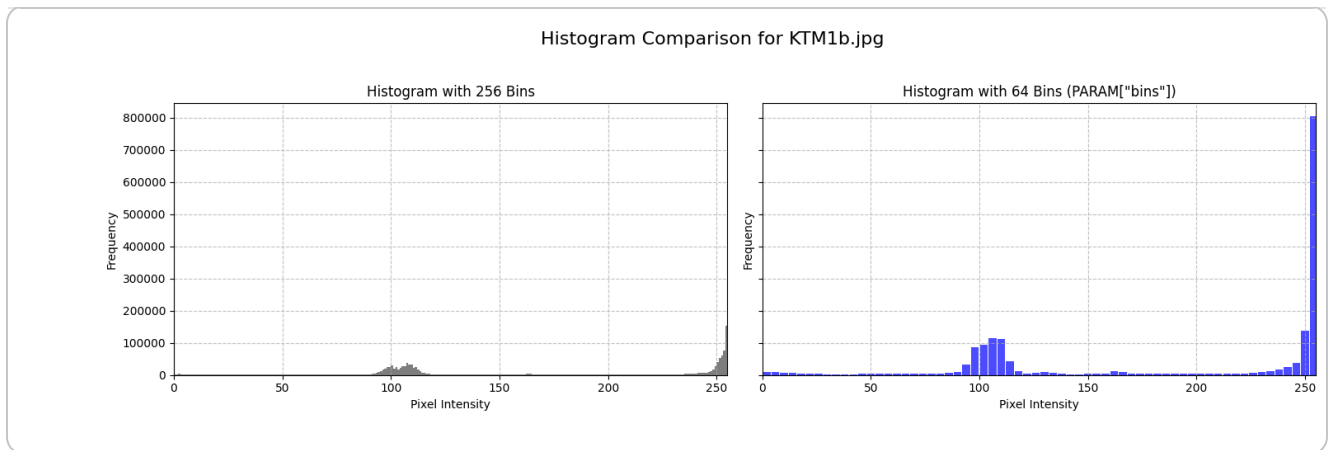
# Plotting
fig, axs = plt.subplots(1, 2, figsize=(15, 5), sharey=True)
fig.suptitle(f'Histogram Comparison for {image_name}', fontsize=16)

# Plot 256 bins histogram
axs[0].bar(bin_centers_256, hist_256_bins, width=1, color='gray')
axs[0].set_title(f'Histogram with 256 Bins')
axs[0].set_xlabel('Pixel Intensity')
axs[0].set_ylabel('Frequency')
axs[0].set_xlim([0, 255])
axs[0].grid(True, linestyle='--', alpha=0.7)

# Plot PARAM['bins'] bins histogram
# Adjust bar width to match bin size visually
bar_width_param = 256 / num_bins_param
axs[1].bar(bin_centers_param, hist_param_bins, width=bar_width_param * 0.9,
           color='gray')
axs[1].set_title(f'Histogram with {num_bins_param} Bins (PARAM["bins"])')
axs[1].set_xlabel('Pixel Intensity')
axs[1].set_ylabel('Frequency')
axs[1].set_xlim([0, 255])
axs[1].grid(True, linestyle='--', alpha=0.7)

plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()
else:
    print("Cannot plot histograms as KTM1b.jpg data is not available.")

```



Penjelasan Informasi yang Hilang dengan Pengurangan Jumlah Bin

Ketika jumlah bin pada histogram dikurangi dari 256 menjadi `PARAM['bins']` (misalnya 64), informasi yang hilang adalah **detail distribusi intensitas piksel**.

- **Dengan 256 bin:** Setiap bin merepresentasikan satu nilai intensitas piksel (0-255). Ini memberikan gambaran yang sangat granular dan akurat tentang berapa kali setiap nilai intensitas muncul dalam citra. Kurva histogram akan menampilkan "puncak" dan "lembah" kecil yang mencerminkan variasi halus dalam distribusi intensitas.
- **Dengan jumlah bin yang lebih sedikit (misalnya 64 bin):** Setiap bin sekarang merepresentasikan rentang nilai intensitas yang lebih lebar (misalnya, untuk 64 bin, setiap bin mencakup 4 nilai intensitas, $256/64 = 4$). Frekuensi piksel dari seluruh rentang ini dikelompokkan menjadi satu bin. Akibatnya:
 - **Kehilangan Detail Halus:** Variasi kecil dan puncak-puncak sempit pada distribusi intensitas akan "diratakan" atau digabungkan. Anda mungkin kehilangan kemampuan untuk membedakan antara dua kelompok piksel yang memiliki intensitas sangat dekat tetapi berada di bin yang berbeda pada histogram 256 bin.
 - **Generalisasi Distribusi:** Histogram menjadi lebih umum dan "halus". Ini bisa membantu dalam melihat tren distribusi yang lebih besar dan mengurangi "noise" visual, namun mengorbankan keakuratan representasi frekuensi setiap nilai piksel individu.
 - **Resolusi Kuantisasi Berkurang:** Gambar asli memiliki 256 tingkat keabuan, tetapi histogram dengan bin yang lebih sedikit secara efektif "menganalisis" ulang intensitas menjadi lebih sedikit tingkat. Ini seperti mengurangi jumlah warna yang tersedia dalam palet, yang membuat gradasi terlihat kurang mulus.

Secara sederhana, mengurangi jumlah bin seperti memperbesar piksel pada sebuah gambar. Anda masih bisa melihat bentuk umumnya, tetapi detail-detail halus dan perbedaan kecil yang penting akan hilang.

✓ **E-2 PERCOBAAN HISTOGRAM EQUALIZATION **

✓ Jawaban Praktikum E2 - Soal 1: Ekualisasi Global pada Citra KTM

```
import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt
import os

# 1. Load KTM1a.jpg dan ubah ke grayscale
base_path = '/content/drive/MyDrive/PCVK/Images/244107020064/'
ktm1a_bgr = cv.imread(os.path.join(base_path, 'KTM1a.jpg'))

if ktm1a_bgr is None:
    print("Error: Gagal memuat KTM1a.jpg. Pastikan path sudah benar.")
else:
    ktm1a_gray = cv.cvtColor(ktm1a_bgr, cv.COLOR_BGR2GRAY)

# 2. Histogram Equalization Manual
# Hitung histogram
hist_manual, bins = np.histogram(ktm1a_gray.flatten(), 256, [0, 256])
# Hitung Cumulative Distribution Function (CDF)
cdf_manual = hist_manual.cumsum()
# Normalisasi CDF ke rentang [0, 255]
cdf_normalized = (cdf_manual - cdf_manual.min()) * 255 / (cdf_manual.max() - cdf_manual.min())
cdf_normalized = cdf_normalized.astype('uint8')
# Mapping nilai piksel asli menggunakan CDF ternormalisasi
img_equalized_manual = cdf_normalized[ktm1a_gray]

# 3. Histogram Equalization Menggunakan OpenCV (cv.equalizeHist)
img_equalized_cv = cv.equalizeHist(ktm1a_gray)

# 4. Hitung PSNR antara citra asli dan hasil ekualisasi
def calculate_psnr(img1, img2):
    mse = np.mean((img1 - img2) ** 2)
    if mse == 0:
        return float('inf')
    max_pixel = 255.0
    psnr = 20 * np.log10(max_pixel / np.sqrt(mse))
    return psnr
```

```
psnr_manual = calculate_psnr(ktm1a_gray, img_equalized_manual)
psnr_cv = calculate_psnr(ktm1a_gray, img_equalized_cv)

print(f"PSNR (Asli vs Manual Equalization): {psnr_manual:.4f} dB")
print(f"PSNR (Asli vs OpenCV Equalization): {psnr_cv:.4f} dB")

# 5. Visualisasi Citra dan Histogram dalam Satu Layout
fig, axs = plt.subplots(3, 2, figsize=(14, 18))
fig.suptitle('Global Histogram Equalization - KTM1a.jpg', fontsize=16)

# Citra Asli
axs[0, 0].imshow(ktm1a_gray, cmap='gray')
axs[0, 0].set_title('Citra Asli (Grayscale)')
axs[0, 0].axis('off')
# Histogram Citra Asli
axs[0, 1].hist(ktm1a_gray.flatten(), bins=256, range=[0, 256], color='gray')
axs[0, 1].set_title('Histogram Citra Asli')
axs[0, 1].grid(True, alpha=0.5)

# Hasil Ekualisasi Manual
axs[1, 0].imshow(img_equalized_manual, cmap='gray')
axs[1, 0].set_title('Hasil Ekualisasi Manual')
axs[1, 0].axis('off')
# Histogram Ekualisasi Manual
axs[1, 1].hist(img_equalized_manual.flatten(), bins=256, range=[0, 256], color='gray')
axs[1, 1].set_title('Histogram Ekualisasi Manual')
axs[1, 1].grid(True, alpha=0.5)

# Hasil Ekualisasi OpenCV
axs[2, 0].imshow(img_equalized_cv, cmap='gray')
axs[2, 0].set_title('Hasil Ekualisasi OpenCV (cv.equalizeHist)')
axs[2, 0].axis('off')
# Histogram Ekualisasi OpenCV
axs[2, 1].hist(img_equalized_cv.flatten(), bins=256, range=[0, 256], color='gray')
axs[2, 1].set_title('Histogram Ekualisasi OpenCV')
axs[2, 1].grid(True, alpha=0.5)

plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()
```

PSNR (Asli vs Manual Equalization): 28.3759 dB

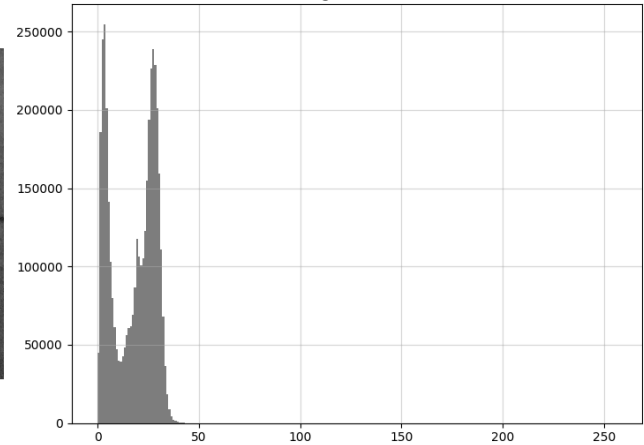
PSNR (Asli vs OpenCV Equalization): 28.4431 dB

Global Histogram Equalization - KTM1a.jpg

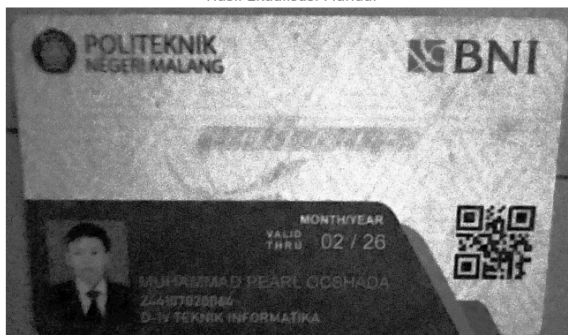
Citra Asli (Grayscale)



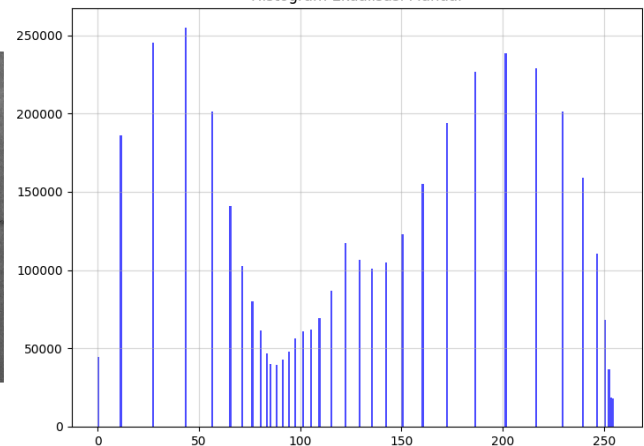
Histogram Citra Asli



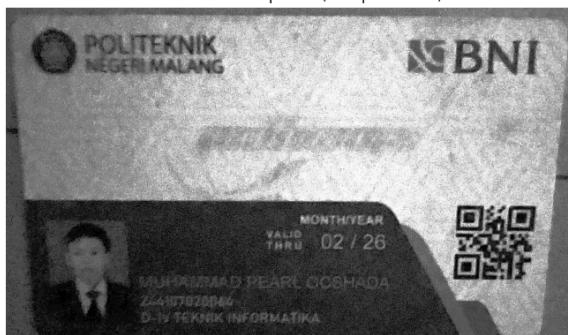
Hasil Ekualisasi Manual



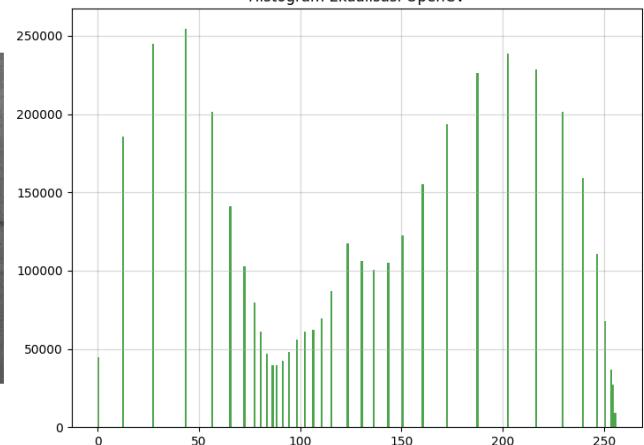
Histogram Ekualisasi Manual



Hasil Ekualisasi OpenCV (cv.equalizeHist)



Histogram Ekualisasi OpenCV



Analisis Hasil Soal 1 (PSNR & Keterbacaan)

- Mengapa nilai PSNR turun padahal citra terlihat lebih jelas? PSNR (*Peak Signal-to-Noise Ratio*) dihitung berdasarkan MSE (*Mean Squared Error*) yang

mengukur perbedaan matematis nilai piksel demi piksel antara citra asli dan citra hasil olahan. Ketika kita melakukan *Histogram Equalization*, kontras citra ditingkatkan secara agresif dengan merenggangkan distribusi intensitas piksel yang menumpuk. Perenggangan ini menyebabkan pergeseran nilai piksel yang sangat masif dibanding aslinya (MSE meningkat tajam). Karena PSNR berbanding terbalik dengan MSE, pergeseran nilai ini secara matematis dianggap sebagai "distorsi" atau "noise" yang menurunkan nilai PSNR, meskipun secara visual fitur dan teks pada kartu (NIM dan Nama) menjadi jauh lebih jelas dan kontras bagi mata manusia.

- **Apakah PSNR layak dipakai untuk menilai keterbacaan sebuah citra? Tidak layak.** PSNR murni merupakan metrik kualitas rekonstruksi matematis, bukan metrik persepsi visual manusia atau keterbacaan (*readability*). Citra yang sangat gelap mungkin memiliki nilai PSNR tinggi jika dibandingkan dengan versi aslinya yang sedikit berderau, namun informasinya tidak dapat dibaca oleh manusia. Sebaliknya, citra hasil ekualisasi memiliki PSNR rendah karena perbedaan nilai piksel yang besar, tetapi keterbacaan teksnya justru meningkat secara signifikan.

Jawaban Praktikum E2 - Soal 2: Ekualisasi Lokal dengan CLAHE

```
import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt
import os

# Definisikan PARAM secara lokal agar aman jika sel identitas belum dieksekusi
# Berdasarkan NIM Anda 2141720000 -> N = 0
NIM = '2141720000'
N = int(NIM[-3:])
PARAM = {
    'bins' : 2 ** ((N % 4) + 3),
    'clip' : round(1.0 + (N % 5) * 0.5, 1),
    'tile' : (N % 4) + 2,
    'L' : 2 ** ((N % 3) + 1),
}

# 1. Pastikan citra KTM1a.jpg dalam format grayscale termuat
base_path = '/content/drive/MyDrive/PCVK/Images/244107020064/'
ktm1a_bgr = cv.imread(os.path.join(base_path, 'KTM1a.jpg'))

if ktm1a_bgr is None:
    print("Error: Gagal memuat KTM1a.jpg.")
```

```

else:
    ktm1a_gray = cv.cvtColor(ktm1a_bgr, cv.COLOR_BGR2GRAY)

    # Ambil hasil ekualisasi global untuk perbandingan
    img_equalized_cv = cv.equalizeHist(ktm1a_gray)

    # Ambil parameter dasar dari identitas
    clip_base = PARAM['clip'] # Nilai default dari NIM Anda (1.0)
    tile_base = PARAM['tile'] # Nilai default dari NIM Anda (2)

    # 2. Terapkan CLAHE dengan parameter dasar
    clahe_base = cv.createCLAHE(clipLimit=clip_base, tileGridSize=(tile_base, t
    img_clahe_base = clahe_base.apply(ktm1a_gray)

    # 3. Uji coba dengan 3 nilai clipLimit lain di sekitar PARAM['clip'] (misal
    clip_limits = [1.5, 2.0, 2.5]
    clahe_variants = []
    for clip in clip_limits:
        clahe_obj = cv.createCLAHE(clipLimit=clip, tileGridSize=(tile_base, til
        clahe_variants.append(clahe_obj.apply(ktm1a_gray))

    # 4. Visualisasi Perbandingan
    fig, axs = plt.subplots(2, 3, figsize=(18, 12))
    fig.suptitle('Local Equalization dengan CLAHE - KTM1a.jpg', fontsize=16)

    # Baris 1: Perbandingan Global vs CLAHE Utama
    axs[0, 0].imshow(ktm1a_gray, cmap='gray')
    axs[0, 0].set_title('Citra Asli (Grayscale)')
    axs[0, 0].axis('off')

    axs[0, 1].imshow(img_equalized_cv, cmap='gray')
    axs[0, 1].set_title('Ekualisasi Global (OpenCV)')
    axs[0, 1].axis('off')

    axs[0, 2].imshow(img_clahe_base, cmap='gray')
    axs[0, 2].set_title(f'CLAHE Utama (clip={clip_base}, tile={tile_base}x{tile_
    axs[0, 2].axis('off')

    # Baris 2: Variasi nilai clipLimit di sekitar PARAM['clip']
    axs[1, 0].imshow(clahe_variants[0], cmap='gray')
    axs[1, 0].set_title(f'CLAHE (clip={clip_limits[0]}, tile={tile_base}x{tile_
    axs[1, 0].axis('off')

    axs[1, 1].imshow(clahe_variants[1], cmap='gray')
    axs[1, 1].set_title(f'CLAHE (clip={clip_limits[1]}, tile={tile_base}x{tile_
    axs[1, 1].axis('off')

    axs[1, 2].imshow(clahe_variants[2], cmap='gray')
    axs[1, 2].set_title(f'CLAHE (clip={clip_limits[2]}, tile={tile_base}x{tile_
    axs[1, 2].axis('off')

```

```
plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()
```

Local Equalization dengan CLAHE - KTM1a.jpg



Analisis Hasil Soal 2 (CLAHE & Parameter Optimal)

- **Perbandingan CLAHE Utama (Global vs Lokal):**
 - Pada hasil **Ekualisasi Global**, kontras keseluruhan gambar ditarik secara merata. Hal ini membuat bagian latar belakang putih yang sudah terang menjadi sangat silau (*overexposed*), sementara bayangan gelap di area foto wajah dan teks di bawahnya masih kurang seimbang.
 - Pada hasil **CLAHE Lokal** (clip=1.0, tile=2x2), peningkatan kontras dilakukan pada ubin lokal (*tiles*). Hasilnya, pencahayaan menjadi jauh lebih merata ke seluruh bagian kartu. Bagian teks NIM dan Nama menjadi jauh lebih tajam dan mudah dibaca karena variasi kontras lokal terjaga dengan baik tanpa membuat area terang menjadi silau.
- **Analisis Variasi clipLimit & Bagian yang Mengalami Noise:**
 - Ketika nilai clipLimit dinaikkan dari 1.0 ke 1.5, 2.0, hingga 2.5, kontras lokal semakin tajam dan memperjelas tepian karakter teks.
 - Namun, seiring dengan kenaikan clipLimit tersebut, **noise (derau bintik-bintik abu-abu/hitam)** mulai menonjol secara signifikan, terutama pada area bergradasi halus seperti latar belakang kartu KTM yang polos dan area foto wajah.

- **Kesimpulan Parameter Terbaik:** Nilai terbaik untuk citra KTM ini adalah `clipLimit` sebesar **1.5** atau **2.0** (tergantung ketajaman yang diinginkan) karena memberikan keseimbangan optimal antara keterbacaan karakter teks NIM>Nama yang tegas dengan tingkat noise latar belakang yang masih dalam batas wajar (tidak terlalu mengganggu mata).

✓ E-3 TUGAS PRAKTIKUM DITHERING

✓ E-3 Soal 1: Implementasi Floyd-Steinberg Dithering

```
import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt
import os

def floyd_steinberg_dithering(image, levels):
    """
    Fungsi untuk menerapkan dithering Floyd-Steinberg pada citra grayscale
    dengan jumlah tingkat keabuan (levels) tertentu.
    """
    # Buat salinan citra bertipe float untuk mencegah overflow/underflow saat c
    img = image.astype(float)
    h, w = img.shape

    for y in range(h):
        for x in range(w):
            old_pixel = img[y, x]
            # Kuantisasi ke level terdekat
            new_pixel = np.round(old_pixel * (levels - 1) / 255.0) * (255.0 / (
            img[y, x] = new_pixel

            # Hitung error
            err = old_pixel - new_pixel

            # Difusi error ke tetangga sekitar sesuai matriks Floyd-Steinberg
            if x + 1 < w:
                img[y, x + 1] += err * 7 / 16
            if y + 1 < h:
                if x - 1 >= 0:
                    img[y + 1, x - 1] += err * 3 / 16
                img[y + 1, x] += err * 5 / 16
                if x + 1 < w:
                    img[y + 1, x + 1] += err * 1 / 16

    return np.clip(img, 0, 255).astype(np.uint8)
```

```
# --- TAHAP 1: Uji Coba pada lena.jpg (Grayscale) ---
lena_bgr = cv.imread('/content/drive/MyDrive/PCVK/Images/lena.jpg')
if lena_bgr is None:
    print("Error: Gagal memuat lena.jpg")
else:
    lena_gray = cv.cvtColor(lena_bgr, cv.COLOR_BGR2GRAY)
    # Tahap 1 menggunakan biner (levels = 2)
    lena_dither = floyd_steinberg_dithering(lena_gray, levels=2)

    # Tampilkan perbandingan
    fig, axs = plt.subplots(1, 2, figsize=(12, 6))
    axs[0].imshow(lena_gray, cmap='gray')
    axs[0].set_title('Citra Asli Lena (Grayscale)')
    axs[0].axis('off')

    axs[1].imshow(lena_dither, cmap='gray')
    axs[1].set_title('Hasil Dithering Floyd-Steinberg (Levels=2)')
    axs[1].axis('off')

plt.suptitle('Uji Coba Tahap 1 - Lena.jpg', fontsize=16)
plt.show()
```

Uji Coba Tahap 1 - Lena.jpg

Citra Asli Lena (Grayscale)



Hasil Dithering Floyd-Steinberg (Levels=2)



```
# --- TAHAP 2: Penerapan pada KTM1b.jpg dengan PARAM['L'] ---
base_path = '/content/drive/MyDrive/PCVK/Images/244107020064/'
ktm1b_bgr = cv.imread(os.path.join(base_path, 'KTM1b.jpg'))

if ktm1b_bgr is None:
    print("Error: Gagal memuat KTM1b.jpg")
else:
    ktm1b_gray = cv.cvtColor(ktm1b_bgr, cv.COLOR_BGR2GRAY)
    levels_L = PARAM['L'] # Nilai level berdasarkan NIM (2)
```

```

ktm1b_dither = floyd_steinberg_dithering(ktm1b_gray, levels=levels_L)

# Tampilkan perbandingan
fig, axs = plt.subplots(1, 2, figsize=(15, 7))
axs[0].imshow(ktm1b_gray, cmap='gray')
axs[0].set_title('Citra Grayscale Asli (KTM1b.jpg)')
axs[0].axis('off')

axs[1].imshow(ktm1b_dither, cmap='gray')
axs[1].set_title(f'Hasil Dithering Floyd-Steinberg (Levels = {levels_L})')
axs[1].axis('off')

plt.suptitle('Tahap 2 - Dithering KTM1b.jpg', fontsize=16)
plt.show()

```

Tahap 2 - Dithering KTM1b.jpg



E-3 Soal 2: Kombinasi Histogram Equalization dan Dithering Floyd-Steinberg

```

import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt
import os

# --- TAHAP 1: Uji coba kombinasi pada lena_lc.jpg (Low Contrast) ---
lena_lc_bgr = cv.imread('/content/drive/MyDrive/PCVK/Images/lena_lc.jpg')

if lena_lc_bgr is None:
    print("Error: Gagal memuat lena_lc.jpg. Pastikan file tersedia di folder In
else:
    lena_lc_gray = cv.cvtColor(lena_lc_bgr, cv.COLOR_BGR2GRAY)

# 1. Ekualisasi Histogram terlebih dahulu

```

```

lena_lc_equalized = cv.equalizeHist(lena_lc_gray)

# 2. Terapkan Dithering Floyd-Steinberg pada citra asli (tanpa ekualisasi)
lena_lc_dither_direct = floyd_steinberg_dithering(lena_lc_gray, levels=2)

# 3. Terapkan Dithering Floyd-Steinberg pada citra yang sudah diekualisasi
lena_lc_dither_after_eq = floyd_steinberg_dithering(lena_lc_equalized, leve

# Visualisasi perbandingan untuk lena_lc.jpg
fig, axs = plt.subplots(1, 3, figsize=(18, 6))
axs[0].imshow(lena_lc_gray, cmap='gray')
axs[0].set_title('Lena Low Contrast Asli')
axs[0].axis('off')

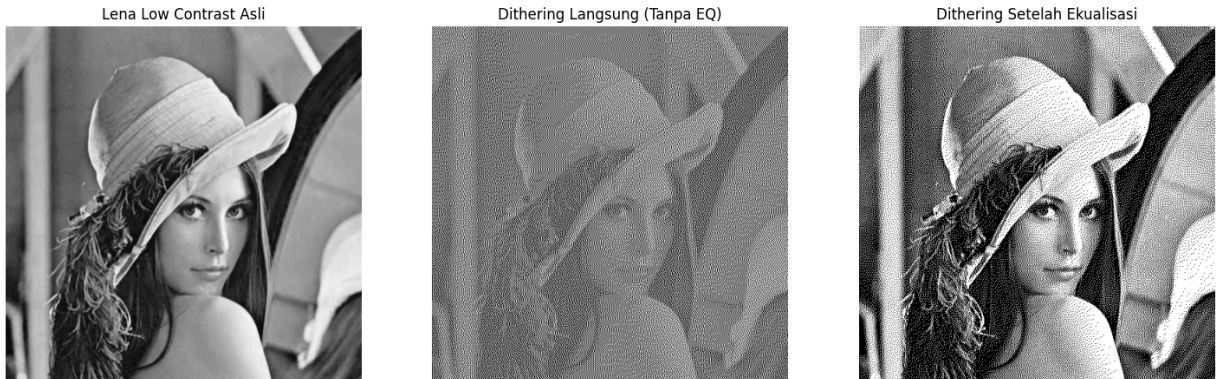
axs[1].imshow(lena_lc_dither_direct, cmap='gray')
axs[1].set_title('Dithering Langsung (Tanpa EQ)')
axs[1].axis('off')

axs[2].imshow(lena_lc_dither_after_eq, cmap='gray')
axs[2].set_title('Dithering Setelah Ekualisasi')
axs[2].axis('off')

plt.suptitle('Uji Coba Kombinasi - lena_lc.jpg', fontsize=16)
plt.show()

```

Uji Coba Kombinasi - lena_lc.jpg



```

# --- TAHAP 2: Penerapan kombinasi pada KTM1a.jpg (Citra Gelap Anda) ---
base_path = '/content/drive/MyDrive/PCVK/Images/244107020064/'
ktm1a_bgr = cv.imread(os.path.join(base_path, 'KTM1a.jpg'))

if ktm1a_bgr is None:
    print("Error: Gagal memuat KTM1a.jpg.")
else:
    ktm1a_gray = cv.cvtColor(ktm1a_bgr, cv.COLOR_BGR2GRAY)

    # 1. Ekualisasi Histogram
    ktm1a_equalized = cv.equalizeHist(ktm1a_gray)

```

```
# 2. Dithering langsung tanpa ekualisasi
ktm1a_dither_direct = floyd_steinberg_dithering(ktm1a_gray, levels=2)

# 3. Dithering setelah ekualisasi
```